Python Code

```python
#!/usr/bin/env python3
"""
Walmart Shipping Data Database Populator

This script processes shipping data from multiple CSV files and populates
a SQLite database with normalized product and shipment information.

Algorithm Overview:
1. Process shipping_data_0.csv (already normalized, direct insert)
2. Process shipping_data_1.csv and shipping_data_2.csv together:
   - Group by shipment_identifier
   - Count product quantities per shipment
   - Join with origin/destination data
   - Insert into database

Author: Walmart Global Tech - Software Engineering Team
"""

import sqlite3
import csv
from pathlib import Path
from collections import defaultdict, Counter
from typing import Dict, List, Tuple, Optional


# Configuration
DATA_DIR = Path("data")
DATABASE_PATH = Path("shipment_database.db")

SHIPPING_DATA_0 = DATA_DIR / "shipping_data_0.csv"
SHIPPING_DATA_1 = DATA_DIR / "shipping_data_1.csv"
SHIPPING_DATA_2 = DATA_DIR / "shipping_data_2.csv"


class DatabasePopulator:
    """Handles all database operations for populating shipping data."""

    def __init__(self, db_path: Path):
        """
        Initialize database connection.

        Args:
            db_path: Path to the SQLite database file
        """
        self.db_path = db_path
        self.conn: Optional[sqlite3.Connection] = None
        self.cursor: Optional[sqlite3.Cursor] = None
        self.product_cache: Dict[str, int] = {}

    def connect(self) -> None:
        """Establish database connection and prepare product cache."""
        try:
            self.conn = sqlite3.connect(self.db_path)
            self.cursor = self.conn.cursor()
            self._load_product_cache()
            print(f"[OK] Connected to database: {self.db_path}")
        except sqlite3.Error as e:
            raise RuntimeError(f"Failed to connect to database: {e}")
```

```python

    def _load_product_cache(self) -> None:
        """Load existing products into cache to avoid duplicates."""
        self.cursor.execute("SELECT id, name FROM product")
        self.product_cache = {name: product_id for product_id, name in self.cursor.fetchall()}
        print(f"[OK] Loaded {len(self.product_cache)} existing products into cache")

    def get_or_create_product(self, product_name: str) -> int:
        """
        Get existing product ID or create new product.

        Args:
            product_name: Name of the product

        Returns:
            Product ID
        """
        if product_name in self.product_cache:
            return self.product_cache[product_name]

        # Insert new product
        self.cursor.execute(
            "INSERT INTO product (name) VALUES (?)",
            (product_name,)
        )
        product_id = self.cursor.lastrowid
        self.product_cache[product_name] = product_id

        return product_id

    def insert_shipment(self, product_id: int, quantity: int,
                        origin: str, destination: str) -> None:
        """
        Insert a shipment record.

        Args:
            product_id: Foreign key to product table
            quantity: Number of products in shipment
            origin: Origin warehouse identifier
            destination: Destination store identifier
        """
        self.cursor.execute(
            """INSERT INTO shipment (product_id, quantity, origin, destination)
               VALUES (?, ?, ?, ?)""",
            (product_id, quantity, origin, destination)
        )

    def commit(self) -> None:
        """Commit the current transaction."""
        if self.conn:
            self.conn.commit()

    def close(self) -> None:
        """Close database connection."""
        if self.cursor:
            self.cursor.close()
        if self.conn:
            self.conn.close()
        print("[OK] Database connection closed")


def load_csv(file_path: Path) -> List[Dict[str, str]]:
```

```
    """
    Load CSV file and return list of row dictionaries.

    Args:
        file_path: Path to CSV file

    Returns:
        List of dictionaries, one per row

    Raises:
        FileNotFoundError: If CSV file doesn't exist
        ValueError: If CSV file is malformed
    """
    if not file_path.exists():
        raise FileNotFoundError(f"CSV file not found: {file_path}")

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            data = list(reader)
        print(f"[OK] Loaded {len(data)} rows from {file_path.name}")
        return data
    except csv.Error as e:
        raise ValueError(f"Error reading CSV file {file_path}: {e}")


def process_shipping_data_0(db: DatabasePopulator, data: List[Dict[str, str]]) -> int:
    """
    Process shipping_data_0.csv - already normalized data.

    This file contains complete shipment records with product, quantity,
    origin, and destination already specified per row.

    Args:
        db: Database populator instance
        data: List of row dictionaries from CSV

    Returns:
        Number of shipments inserted
    """
    shipments_inserted = 0

    for row in data:
        product_name = row['product']
        quantity = int(row['product_quantity'])
        origin = row['origin_warehouse']
        destination = row['destination_store']

        # Get or create product and insert shipment
        product_id = db.get_or_create_product(product_name)
        db.insert_shipment(product_id, quantity, origin, destination)
        shipments_inserted += 1

    db.commit()
    print(f"[OK] Processed shipping_data_0.csv: {shipments_inserted} shipments inserted")
    return shipments_inserted


def process_shipping_data_1_and_2(db: DatabasePopulator,
                                  data1: List[Dict[str, str]],
                                  data2: List[Dict[str, str]]) -> int:
    """
```

```
    Process shipping_data_1.csv and shipping_data_2.csv together.

    Data1 contains one product per row with shipment_identifier.
    Multiple rows with same shipment_identifier represent one shipment.
    Data2 contains origin/destination for each shipment_identifier.

    Algorithm:
    1. Group data1 by shipment_identifier
    2. Count product occurrences per shipment (quantity = count)
    3. Create lookup dict from data2 for origin/destination
    4. Join and insert shipments

    Args:
        db: Database populator instance
        data1: Rows from shipping_data_1.csv
        data2: Rows from shipping_data_2.csv

    Returns:
        Number of shipments inserted
    """
    # Build shipment metadata lookup from data2
    shipment_metadata = {}
    for row in data2:
        shipment_id = row['shipment_identifier']
        shipment_metadata[shipment_id] = {
            'origin': row['origin_warehouse'],
            'destination': row['destination_store']
        }

    # Group data1 by shipment_identifier and count products
    shipment_products = defaultdict(list)
    for row in data1:
        shipment_id = row['shipment_identifier']
        product_name = row['product']
        shipment_products[shipment_id].append(product_name)

    # Process each shipment
    shipments_inserted = 0
    for shipment_id, products in shipment_products.items():
        # Get origin and destination for this shipment
        if shipment_id not in shipment_metadata:
            print(f"[WARNING] No metadata found for shipment {shipment_id}, skipping")
            continue

        metadata = shipment_metadata[shipment_id]
        origin = metadata['origin']
        destination = metadata['destination']

        # Count quantity of each unique product in this shipment
        product_counts = Counter(products)

        # Insert one shipment record per unique product
        for product_name, quantity in product_counts.items():
            product_id = db.get_or_create_product(product_name)
            db.insert_shipment(product_id, quantity, origin, destination)
            shipments_inserted += 1

    db.commit()
    print(f"[OK] Processed shipping_data_1.csv + shipping_data_2.csv: {shipments_inserted} shipments
inserted")
    return shipments_inserted

```

```python

def main() -> None:
    """
    Main execution function.

    Orchestrates the entire database population process:
    1. Load all CSV files
    2. Connect to database
    3. Process shipping_data_0.csv
    4. Process shipping_data_1.csv and shipping_data_2.csv
    5. Report final statistics
    """
    print("=" * 70)
    print("Walmart Shipping Data Database Populator")
    print("=" * 70)
    print()

    db = None

    try:
        # Load CSV files
        print("Step 1: Loading CSV files...")
        data_0 = load_csv(SHIPPING_DATA_0)
        data_1 = load_csv(SHIPPING_DATA_1)
        data_2 = load_csv(SHIPPING_DATA_2)
        print()

        # Connect to database
        print("Step 2: Connecting to database...")
        db = DatabasePopulator(DATABASE_PATH)
        db.connect()
        print()

        # Process shipping_data_0.csv
        print("Step 3: Processing shipping_data_0.csv...")
        shipments_0 = process_shipping_data_0(db, data_0)
        print()

        # Process shipping_data_1.csv and shipping_data_2.csv
        print("Step 4: Processing shipping_data_1.csv and shipping_data_2.csv...")
        shipments_1_2 = process_shipping_data_1_and_2(db, data_1, data_2)
        print()

        # Final statistics
        total_shipments = shipments_0 + shipments_1_2
        total_products = len(db.product_cache)

        print("=" * 70)
        print("Database Population Complete!")
        print("=" * 70)
        print(f"Total products in database: {total_products}")
        print(f"Total shipments inserted: {total_shipments}")
        print(f"  - From shipping_data_0.csv: {shipments_0}")
        print(f"  - From shipping_data_1.csv + shipping_data_2.csv: {shipments_1_2}")
        print("=" * 70)

    except FileNotFoundError as e:
        print(f"[ERROR] {e}")
        return
    except ValueError as e:
        print(f"[ERROR] {e}")
        return
```

```
    except RuntimeError as e:
        print(f"[ERROR] {e}")
        return
    except Exception as e:
        print(f"[ERROR] Unexpected error: {e}")
        raise
    finally:
        if db:
            db.close()


if __name__ == "__main__":
    main()

```